

Base de programmation C/C++

Semestre 1 2024/2025

I. Algorithme, variables, conditions et boucles.

Bas et Haut niveau

C
+
+

VS

P
Y
T
H
O
N

Bas niveau

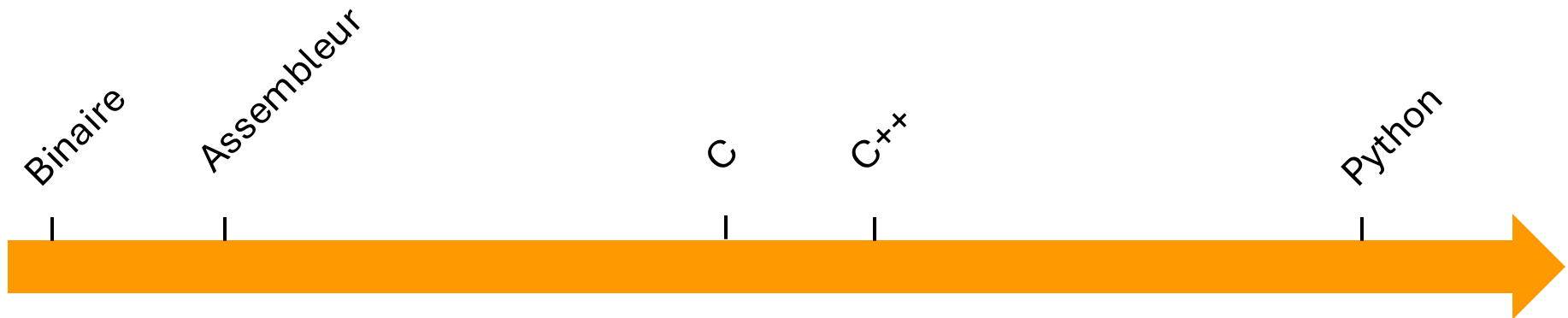
Proche du langage machine /
de la logique des composants

+ de contrôle (ex : la mémoire)

Haut niveau

Proche du langage humain

- + lisible et facile à coder
- + facile à corriger
- + portable



Compilation

C
+
+

vs

P
Y
T
H
O
N

Utilisation d'un compilateur pour :

- Transformer le code source en langage machine exécutable
- Optimiser le code
- Analyse lexicale, syntaxique et sémantique
 - Signale des erreurs (de compilation)
 - Émet des avertissements

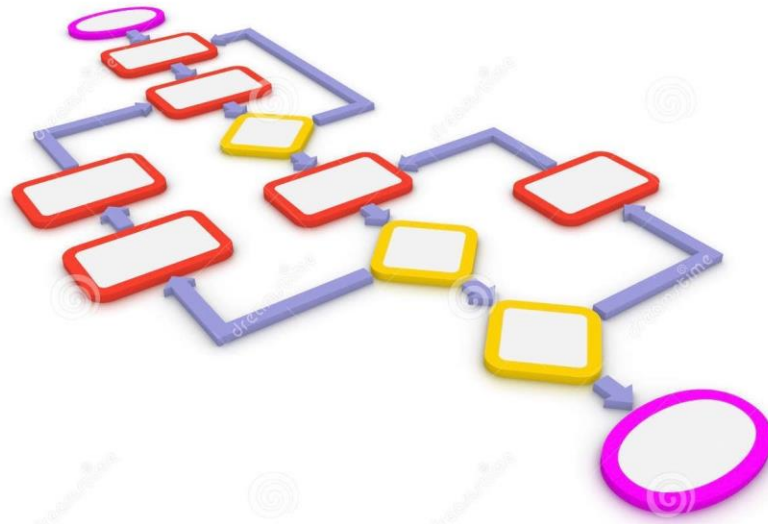


Un algorithme,
c'est quoi ?

Définition



Définition



Un algorithme :

- Est une suite d'instruction
 - Finie
 - Exécutée dans l'ordre
- Prend des entrées
- Produit un résultat

Un bon algorithme est :

- Peu "complexe"
- Rapide
- Peu gourmand en ressources

Exemples



Recette



Monter un
meuble



Invoquer un
démon



Rubik's
Cube



Un itinéraire



Production
industrielle



Tutoriel de
jeu vidéo



Programme de
machine à laver

Représentations

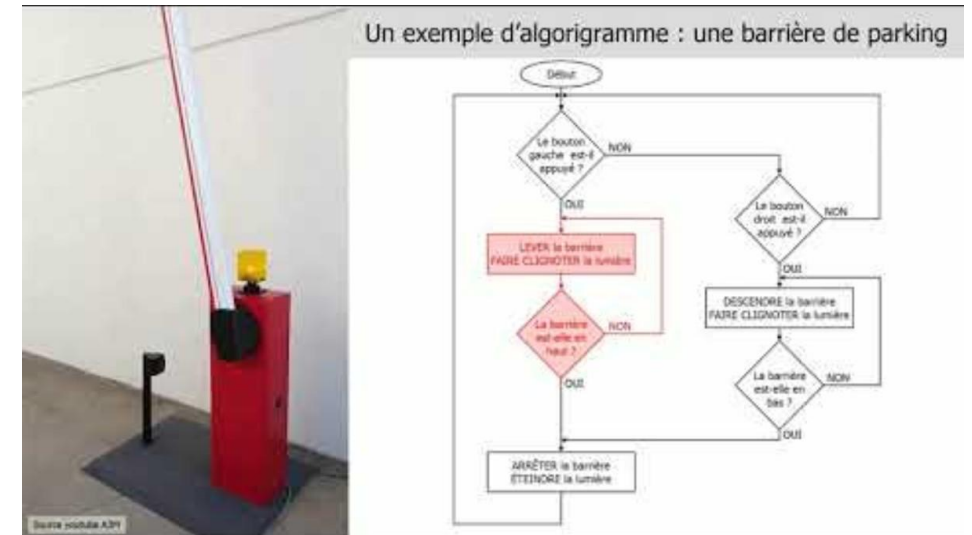
Algorithme 1: Dijkstra: (origine (s), graphe $G(V,E)$):

```

|  $dist(s) \leftarrow 0$ ;
|  $Q \leftarrow \emptyset$ 
| Pour tout nœud  $v$  de  $V$  Faire
|   Si  $v \neq s$ 
|      $dist[v] \leftarrow +\infty$ 
|      $pred[v] \leftarrow []$ ;
|   Fin de si
|    $Q \leftarrow Q \cup \{v\}$ ;
| Fin de Pour
| Tant que  $Q \neq \emptyset$  Faire
|    $u \leftarrow v$  tel que  $v = \min dist[v]$  et  $v \in Q$ 
|    $Q \leftarrow Q \setminus \{u\}$ ;
|   Pour tout voisin  $v$  de  $u$ 
|      $temp \leftarrow dist[u] + c(u,v)$ 
|     Si  $temp < dist[v]$ 
|        $dist[v] \leftarrow temp$ 
|        $pred[v] \leftarrow u$ 
|     Fin de si
|   Fin pour
| Fin de tant que
| Résultats  $dist[], pred[]$ 
| Fin

```

Pseudo-code



Algorithme

Travail d'un programmeur



Je veux un smoothie
fraise/banane vegan

Entrées

**Recette de Smoothie aux Fruits**

2 bananes mûres
1 tasse de fraises fraîches
1/2 tasse de lait d'amande
1 cuillère à soupe de sirop d'agave

Ajouter le sirop d'agave dans le blender. (facultatif)
Ajouter les fruits dans le blender.
Verser le lait d'amande dans le blender.

Mixer 30 secondes.
Vérifier la consistance : elle doit être lisse et
crémeuse.
Sinon remixer à nouveau 30 secondes avant de
vérifier à nouveau la consistance et répéter ce jusqu'à
ce qu'elle soit satisfaisante.

Instructions

Dégustez votre smoothie frais et fruité !



Sortie
attendue

Travail d'un programmeur

Recette de Smoothie aux Fruits

2 bananes mûres
1 tasse de fraises fraîches
1/2 tasse de lait d'amande
1 cuillère à soupe de sirop d'agave

Ajouter le sirop d'agave dans le blender. (facultatif)

Ajouter les fruits dans le blender.

Verser le lait d'amande dans le blender.

Mixer 30 secondes.
Vérifier la consistance : elle doit être lisse et crémeuse.
Sinon remixer à nouveau 30 secondes avant de vérifier à nouveau la consistance et répéter ce jusqu'à ce qu'elle soit satisfaisante.

Dégustez votre smoothie frais et fruité !

Recette de Smoothie aux Fruits

bananes = 2
fraises fraîches = 1 tasse
lait d' amande = 1/2 tasse
Sirop agave = 1 cas

Variables

Condition SI vous avez du sirop d'agave
Ajouter le sirop d'agave dans le blender

Ajouter les fruits dans le blender.
Verser le lait d'amande dans le blender.

Boucle TANT QUE la consistance n'est pas lisse
Boucle POUR 30 secondes
Mixer

Dégustez votre smoothie frais et fruité !



Les variables

Pour quoi faire ?

Sauvegarder temporairement des informations ou résultats pour nous en resservir plus tard.

Comment ?

Stocker les valeurs dans la mémoire.
Puis ensuite les lire, les remplacer ou les effacer.

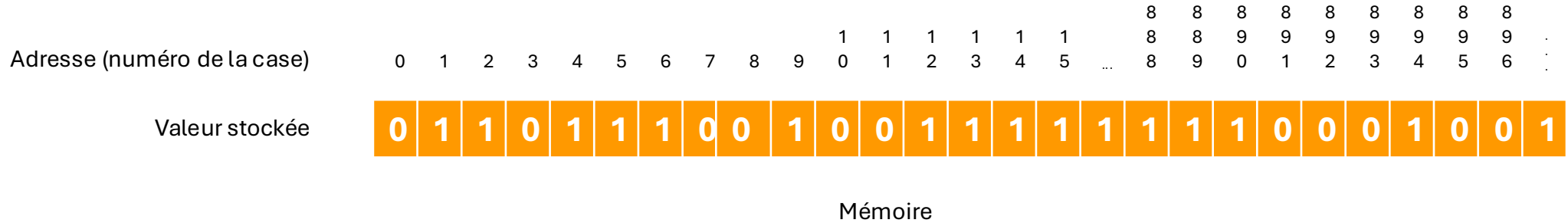
Valeur stockée

0	1	1	0	1	1	1	0	0	1	0	0	1	1	1	1	1	1	1	0	0	0	1	0	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Mémoire

Une variable : un nom, une adresse, un type, une valeur.

Déclarer une variable : Typage



Pourquoi :

- **Combien de bits (cases)** pour stocker la valeur puis la lire ?
- **Ne pas gaspiller de la mémoire** : réserver la mémoire nécessaire mais pas plus.
- **Gestion des erreurs** : ne pas mettre une poire à la place d'une pomme qui était attendue.

Les principaux types (pour le moment) :

- Les entiers : **int** -> 32 bits
- Les décimaux : **double** -> 64 bits
- Les booléens (vrai/faux) : **bool** -> 8 bit

Déclaration

Réserve en mémoire des blocs côte à côte pour stocker l'information.

Obligatoire avant d'utiliser une variable

```
double total_cost2;  
  
int length_array_date;  
  
bool success;
```

Initialisation / affectation



Il y a déjà des valeurs dans la mémoire réservée.

1	1	1	1	1	1	1	1	0	0	0	1	0	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Mémoire

Il faut y mettre une valeur à nous avant de l'utiliser.

```
total_cost2 = 12.5;  
length_array_date = 8;
```

Fonction printf()

Permet d'afficher du texte dans la console

```
printf("bonjour \n");
```

Pour afficher le contenu d'une variable, il faut donner le type de variable puis son nom à la fin

%d = int, %f = double, %d bool

```
int age = 20;
```

```
printf("Age : %d and\n", age)
```

```
double taille = 1.78
```

```
printf("Taille : %f metres\n", taille)
```

```
bool vivant = true;
```

```
printf("Est vivant : %d \n", vivant)
```

Fonction scanf()

Permet de lire des entrées utilisateur

```
scanf("%type", &nom_variable);
```

%d = int, %lf = double, %d bool

```
int valeur;
```

```
printf("Votre nombre entier :")
```

```
scanf("%d", &valeur)
```




Les conditions

Structure et mots clés en C++

Faire suivre différents voies à l'algorithme pour faire des traitements différents selon la valeur de certaines données.

1. *Si*

1. *if*

1. *Si*

1. *if*

2. *Sinon*

2. *else*

1. *Si*

1. *if*

2. *Sinon Si*

2. *else if*

3. *Sinon Si*

3. *else if*

4. ...

4. ...

1. *Si*

1. *if*

2. *Sinon Si*

2. *else if*

3. *Sinon Si*

3. *else if*

4. ...

4. ...

5. *Sinon*

5. *else*

Si *doigt* = 1
"THUMB UP !"

Sinon Si *doigt* = 2
"ON MONTRE PAS DU DOIGT !"

Sinon Si *doigt* = 3
"SOIS POLI !"

Sinon Si *doigt* = 4 OU *doigt* = 5
"ILS SERVENT A RIEN"

Sinon
"HEU... C'EST QUOI CETTE
TENTACULE ?!"

Tests

```
if (condition)
{
    //fait des trucs si la condition est VRAIE !
}
```

Les opérateurs relationnels

- == Egal
- < Inférieur
- <= Inférieur ou égal
- > Supérieur
- >= Supérieur ou égal
- != Différent

```
1 int main()
2 {
3     int a = 10;
4     int b = 20;
5
6     if (a == b)
7     {
8         cout << "a est égal à b" << endl;
9     }
10    else if (a != b)
11    {
12        cout << "a est différent de b" << endl;
13
14        if (a < b)
15        {
16            cout << "a est inférieur à b" << endl;
17        }
18        else if (a > b)
19        {
20            cout << "a est supérieur à b" << endl;
21        }
22    }
23 }
```

Tests multiples

```
if ( condition1 ET condition2 )  
{  
    //fait des trucs si la condition est VRAIE !  
}
```

Les opérateurs logiques

&& ET

|| OU

! NOT (inverse la valeur)

Exemples

```
if ( a > 0 && b > 0 )
```

```
if ( a < 0 || b > 0 )
```

```
if ( a > 0 && (b > 0 || c > 0) )
```

```
if ( a > 0 && ! (b > 0 || c > 0) )
```



Les boucles

While / Tant que : boucle indéfinie

Répète un bloc d'instructions un nombre de fois indéfini, inconnu à l'avance, fixé par une condition.

Répète TANT QUE la condition est vraie. C'est un *if* qui se répète.

Une partie : condition de poursuite

For / Pour : boucle définie

Répète un bloc d'instructions un nombre de fois prédéterminé, défini, connu à l'avance.

Initialisation	On démarre à combien ?
Condition(s) de fin	Tant que c'est bon on continue
Pas de l'itération	On modifie de combien le compteur avant de reboucler ?

```
int i;  
i = 0 ;  
  
while ( i < 10 )  
{  
    i = chiffrageauhasard;  
}
```

```
int i ;  
  
for ( i = 0 ; i < 10 ; i = i +1 )  
{  
    // fait des trucs;  
}
```

La pile (stack)

Organisation LIFO (Last In, First Out)

Type de mémoire rapide

À chaque nouveau bloc -> Une nouvelle zone ("frame") est empliée

Quand on accède au premier élément de la pile, c'est le dernier élément rentré qui sort



Les variables et leur portée

La portée

Zone dans laquelle la variable existe.

Une variable existe dans :

- le bloc dans lequel elle est déclarée
- les blocs fils (bloc imbriqué).

Dès que l'on sort du bloc elle n'existe plus.

```
int main()
{
    int a;
    a = 3;

    int b;
    b = 5;

    if ( a < 5 )
    {
        cout << a << endl;
        int c;
        c = 5;
    }

    cout << c << endl;

    int c;
    c = 4;
}
```

Portée locale

Erreur : c n'existe plus